

Introducing Large Language Models into the Design Flow of Time Sensitive Networking

Rubi Debnath¹, Luxi Zhao², Mohammadreza Barzegaran³, Paul Pop⁴, and Sebastian Steinhorst⁵

^{1,5}TUM School of Computation, Information and Technology, Technical University of Munich, Germany

²Electronic Information Engineering College, Beihang University, Beijing, China

³Department of Electrical Engineering and Computer Science, University of California, Irvine, USA

⁴DTU Compute, Technical University of Denmark, Denmark

^{1,5}firstname.lastname@tum.de, ²zhaoluxi@buaa.edu.cn, ³barzegm1@uci.edu, ⁴paupo@dtu.dk

Abstract—The growing demand for real-time, safety-critical systems has significantly increased both the adoption and complexity of Time Sensitive Networking (TSN). Configuring an optimized TSN network is highly challenging, requiring careful planning, design, analysis, verification, validation, and deployment. Large Language Models (LLMs) have recently demonstrated strong capabilities in solving complex tasks, positioning them as promising candidates for automating end-to-end TSN deployment and management, referred to as TSN orchestration. This paper outlines the steps involved in TSN orchestration and the associated challenges. To assess the capabilities of existing LLMs, we conduct an initial proof-of-concept case study focused on TSN tasks across multiple models. Building on these insights, we propose an LLM-assisted orchestration framework. Unlike prior research on LLMs in computer networks, which has concentrated on general configuration and management, TSN-specific orchestration has not yet been investigated. We present the building blocks for automating TSN using LLMs, describe the proposed pipeline, and analyze opportunities and limitations for real-world deployment. This work provides the first roadmap toward assessing the feasibility of LLM-assisted TSN orchestration.

Index Terms—Time sensitive networking, large language models, generative AI (GenAI), network configuration, automation.

I. INTRODUCTION

Time-Sensitive Networking (TSN) [1] provides deterministic communication and bounded latency, making it essential for safety-critical domains such as industrial automation, vehicular systems, aerospace, healthcare, and emerging 5G-TSN applications. By supporting mixed-criticality traffic [2] on the same network, TSN enables diverse Quality of Service (QoS) requirements to coexist. However, configuring an optimized TSN network remains highly challenging. It involves multiple building blocks (refer to Fig. 1) including planning, design, verification, validation, and deployment, all of which demand deep expertise in TSN. This process is time-consuming, tedious, and heavily dependent on domain experts.

Meanwhile, Large Language Models (LLMs) have achieved rapid adoption across research and industry, demonstrating impressive capabilities in solving complex, multi-step tasks. Prior studies have demonstrated their effectiveness for mathematical reasoning [3], network configuration [4], [5], network

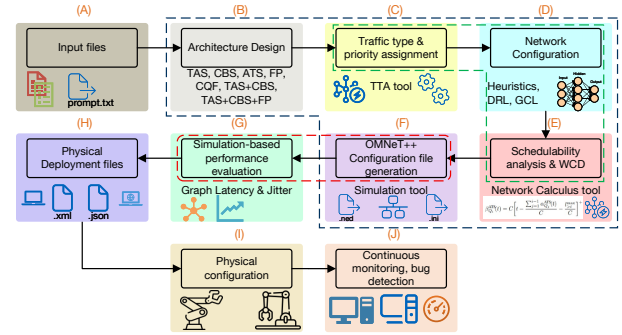


Fig. 1: Proposed LLM-aided TSN Orchestrator architecture. The pipeline illustrates the key steps in the end-to-end orchestration and deployment of TSN networks.

planning [6], and fifth-generation (5G) network analysis [7]. In industry, LLMs are already being explored for automating infrastructure management and operational support. Despite this progress, their application to TSN remains largely unexplored. Unlike general computer networking, TSN introduces unique challenges, including deterministic guarantees, network configuration [1], [8], schedulability analysis [9], traffic type assignment (TTA) [2], [10], simulative evaluation [11], [12] and a lack of open-source tools, datasets and sub-standards. TSN is often used in safety-critical systems, so the outputs of an LLM-assisted orchestrator cannot be judged only by syntactic correctness or convenience. In safety assurance [13], a workflow must provide the properties and evidence needed to justify timing and deployment decisions.

A. Motivation and Contributions

Due to the inherent complexity of TSN and the high level of domain expertise required for deployment, there is a need for alternative approaches that can streamline the end-to-end process. Considering the huge capabilities of LLMs, our goal is to explore their potential for automating key stages of real-world TSN deployment. In this paper, we use the term **TSN orchestration** to refer to the end-to-end pipeline required to configure, validate, and deploy a TSN network (step A to

J of Fig. 1). An LLM may participate in some or all of these steps (shown in Fig. 1), depending on the level of automation desired. The ability of LLM models to process natural language instructions, model configurations, and iteratively refine solutions makes them attractive for handling multi-stage orchestration pipelines. However, no prior work has investigated their use in TSN-specific orchestration. This gap motivates our work to perform the initial evaluation of LLMs with TSN tasks. Our main contributions are:

- We introduce a conceptual framework illustrating how LLMs can support planning, design, verification, validation, and deployment of TSN networks. We identify orchestration stages (refer to Fig. 1) where LLMs can be effective and highlight those that require hybrid integration with external tools.
- We present an illustrative case study across several state-of-the-art models to showcase current capabilities and limitations of the models.
- Lastly, we discuss the challenges, limitations, and future research directions of LLMs in TSN.

Overall, this paper serves as a starting point for the community to explore LLM-assisted orchestration of TSN.

II. BACKGROUND

TSN is a layer-2 mechanism that provides deterministic communication and bounded latency. TSN achieves this by utilizing shaping and scheduling mechanisms, including the Time Aware Shaper (TAS) [12], Credit Based Shaper (CBS) [12], Asynchronous Traffic Shaper (ATS) [9], Cyclic Queuing and Forwarding (CQF) [14], and Frame Preemption (FP) [12].

Furthermore, for the configuration and scheduling of TSN flows in the network, optimized algorithms are required. These algorithms are often developed using mathematical models [1], heuristics [8], and advanced machine learning models [10]. However, developing optimal scheduling and configuration algorithms is NP-hard [8], which requires domain-specific algorithms to reduce time complexity.

The schedulability analysis further provides the mathematical upper bounds of the worst-case delay (WCD) of the TSN flows in the network. For this, Network Calculus (NC) tools [9] are widely used both in academia and industry. This process is widely known as schedulability analysis or WCD calculation. These tools compute upper bounds on delay and jitter, which is particularly crucial for event-triggered (ET) traffic, especially when assessing the impact of interference from co-existing time-triggered (TT) flows.

Since the worst-case bounds derived from NC can be pessimistic, simulation frameworks such as OMNeT++ [12], [14] are often used to complement the analysis. These simulations help assess the system's performance under statistical conditions, providing insights into the practical gap between the theoretical worst-case and observed behavior.

Despite their importance, tools such as NC models, TSN-specific scheduling and configuration algorithms are rarely

open source, limiting broader adoption and automation. Furthermore, all these steps are typically evaluated and optimized individually. Each of these models are inherently complex, requiring domain experts to carefully develop specific algorithms and tools.

A. LLMs in Networking

LLMs have recently demonstrated strong capabilities in solving complex reasoning and multi-step tasks. They excel at parsing natural language, synthesizing structured outputs, and integrating information across diverse sources, making them promising for networking tasks. Recent works [5]–[7] have explored the use of LLMs for various computer networking problems. For example, [7] introduced Mobile-LLaMA, an open-source, mobile-network-specialized LLM designed for 5G network management and data analysis. Huang et al. in [6] proposed ChatNet, a domain-adapted LLM framework that reduces the complexity of network planning tasks. In [5], the authors demonstrated how LLMs can generate protocol-specific configurations by leveraging newly acquired knowledge from the internet. Other works [4], [5] have demonstrated LLM-driven configuration synthesis, protocol-specific configuration generation, and knowledge acquisition from online sources.

Existing TSN orchestration relies heavily on domain-specific algorithms, optimization tools, and custom scripts. These approaches are rigid, and require significant manual engineering. LLMs introduce a new leap by offering a natural-language interface that allows even non-experts to express requirements in plain English (for example, “set up a TSN network with three TSN flows with 1 ms deadlines using TAS”).

B. Research Questions

While prior research has successfully applied LLMs to general networking problems, no existing work has addressed their use for TSN-specific orchestration. TSN introduces additional challenges such as deterministic guarantees, schedulability analysis, and mixed-criticality flow assignment, which remain unexplored in the context of LLMs. This gap motivates our work to outline the steps, workflow, and challenges of developing LLM-assisted TSN orchestration. In this paper, we aim to answer the following research questions:

RQ 1: *Can LLMs help with orchestrating TSN networks?*

RQ 2: *What steps are required for LLMs to effectively orchestrate TSN networks?*

RQ 3: *What are the challenges and future directions of LLM-assisted TSN orchestration?*

As shown in Fig. 1, multiple steps are involved in progressing from initial planning to real-world TSN deployment. We separate these steps and map them into LLM-assisted building blocks. The proposed LLM-assisted orchestration workflow consists of: (A) understanding topology, flow information, and network parameters, (B) architecture and mechanism selection, (C) traffic type and priority assignment, (D) network

configuration or schedule synthesis, (E) schedulability analysis and WCD calculation, (F) OMNeT++ simulation files, (G) simulation-based performance evaluation and verification, (H) physical deployment files, (I) physical configuration, and (J) continuous monitoring and bug detection (shown in Fig. 1).

III. END-TO-END TSN ORCHESTRATION USING LLMs

Unlike general network configuration steps described in [5], TSN orchestration requires several TSN-specific stages as shown in Fig. 1. As discussed, related work focuses on each section of Fig. 1 individually. Therefore, there is currently a lack of research addressing the entire pipeline together. An efficient LLM-based model for TSN should execute a sequence of well-defined tasks to achieve complete end-to-end orchestration. We present a roadmap of these tasks, with the proposed pipeline illustrated in Fig. 1. These tasks have different assurance implications. Input parsing, deployment file generation, and monitoring summaries mainly support engineering productivity. In contrast, traffic assignment, schedule synthesis, schedulability analysis, and physical deployment influence whether timing and safety requirements are satisfied. For this latter group, a one-shot prompt-answer workflow may be insufficient with the current LLM capabilities. Instead, the LLMs should be used as a translation and orchestration layer around solvers, NC engines, simulators, validators, and, where relevant, model checkers or theorem provers, while these tools remain the source of the evidence used in the final decision.

A. Input Parsing

Users may provide either (a) a natural-language description of the topology and flows or (b) structured files containing flow and topology parameters. While natural language is feasible for small scenarios, structured input is more practical for large-scale networks with hundreds of flows. The LLM extracts information from these files and converts it into a machine-readable format. As noted in [15], describing flow properties purely in natural language can lead to information loss. We therefore use a unified structure for describing flows, which the LLM processes for further steps.

B. Architecture Design

TSN supports multiple combinations of shapers and schedulers (e.g., TAS, CBS, CQF, FP). The architecture may be predefined by the system engineer or suggested by the LLM based on QoS requirements. The model iterates over candidate designs, refining choices using validation feedback. At this stage, the LLM determines the mechanisms or TSN architecture to be deployed and explores alternative designs until performance targets are satisfied.

C. Traffic Type and Priority Assignment

The model maps flows to TSN traffic types and priorities. Given a set of flows, if the TSN traffic type and priority are not known or predefined in the user prompt, the model examines

the requirements and description of the prompt and, based on domain knowledge, assigns suitable traffic types and priorities. An efficient model should iteratively refine the traffic type and priority assignment through schedulability and simulation-based performance analysis. *Since TTA is NP-hard, external solvers such as [10] may be integrated to enhance efficiency and accuracy. Furthermore, the architecture given by the LLM should be verified using external solvers.* LLM models are unlikely to achieve optimal accuracy without incorporating external TTA tools.

D. Network Configuration

TAS works based on the predefined static table known as the Gate Control List (GCL), also denoted as the scheduling table. CQF and other cyclic shaper variants require the network configuration and the offset (start time of the flow in the source node) values. These are determined by specific algorithms designed to solve these NP-hard problems. Based on the architecture and the traffic type assigned by the model, the scheduling and configuration mechanisms are derived in this stage. Traditional algorithms are developed using heuristics, metaheuristics, and machine learning models. *LLMs could integrate existing state-of-the-art optimization algorithms for scheduling and configuration. In this hybrid approach, the LLM will automate the algorithm selection (from a bunch of existing algorithms), input preparation, and iterative refinement. This enables to deliver near-optimal solutions with reduced manual effort and lower operational complexity, while maintaining the mathematical guarantees provided by the underlying optimization tools.*

E. Schedulability Analysis

Schedulability analysis verifies whether the flows meet the QoS requirements (such as deadlines) by computing the WCD using NC tools [9]. To perform the schedulability analysis, the network configuration from the previous step is required. *As LLMs are not reliable for precise mathematical reasoning [6], external NC modules are required to compute accurate delay bounds. The LLM prepares the NC inputs by translating natural-language or structured descriptions of flows, priorities, and schedules into NC-compatible formats (such as constraints and equations). The output generated by the scheduling algorithm may not be compatible with the NC model input, and LLM can solve this by adapting the format of the files. It can further interpret and summarize the NC outputs, making them understandable to non-experts.* Furthermore, the LLM feeds the results back into the traffic type and priority configuration for further refinement, as discussed in [10].

F. OMNeT++ Configuration File Generation

Schedulability analysis provides the NC-based WCD analysis of the TSN flows in the network, which informs the theoretical upper bounds of delay and jitter. However, to

achieve a complete performance evaluation of the network, simulation-based evaluation is also required. In traditional approaches, OMNeT++ based performance evaluations can be conducted in parallel with schedulability analysis. We need the network configuration, traffic types, and the priorities of the flows to perform simulation-based evaluation. OMNeT++ configuration files are typically generated using custom Python scripts or manually written by simulation developers. Creating OMNeT++ configuration files manually for very large and complex networks is tedious and error-prone. *Therefore, in this stage, the LLM model generates the OMNeT++ configuration files for validation and performance evaluation. More specifically, we are interested in the .ned and .ini files of OMNeT++, which the model should provide without manual intervention. These files should be generated with the appropriate syntax and structure, which can be learned by the model during training. The model can also perform self-checks to evaluate syntax errors in the generated OMNeT++ files.*

G. Simulation-based Performance Evaluation

Performance evaluation verifies and validates the generated architecture and configurations. In this stage, we refer to simulation-based evaluation using state-of-the-art simulators and frameworks [12], [14]. While syntax errors are addressed earlier, this stage focuses on assessing network performance using simulation frameworks such as OMNeT++ and NS-3. The LLM produces simulation-ready configuration files (e.g., .ned, .ini) in the previous stage, and in this stage we execute the simulations. The simulation can be performed on an external OMNeT++ framework.

H. Physical Deployment Files

Following successful verification and validation of the network, we propose that the LLM produce deployment-ready network configuration files, including routes, VLAN tags, and other TSN-specific configuration files. These deployment files are intended for real-world industrial deployment and are therefore distinct from the OMNeT++ configuration files generated in the previous stage. Prior research has shown that LLMs can generate router configurations [16] efficiently. Consequently, fine-tuned models can be adapted to produce TSN-specific deployment files. *Furthermore, the models can be refined to provide vendor-specific (such as Cisco, Hirschmann, NXP) templates and configuration files. By training on vendor manuals and sample configurations, LLMs can generate vendor-specific deployment files, ensuring compatibility across different platforms.*

I. Physical Configuration

The deployment files are then applied to the industrial or application-specific infrastructure. *At this stage, zero-touch configuration [4] can be adopted to provision and configure the entire network with a single operation, minimizing manual intervention and reducing the potential for human error.*

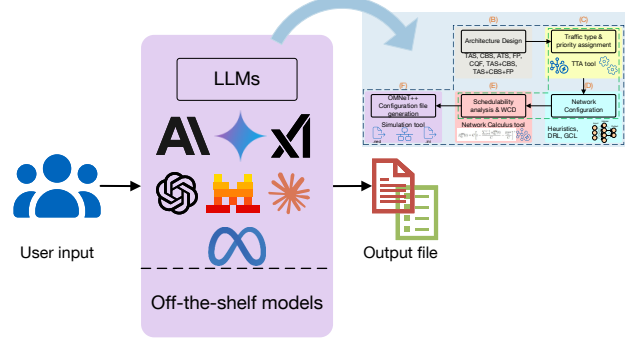


Fig. 2: Overview of the case study setup.

J. Continuous Monitoring and Bug Detection

Continuous monitoring, log analysis, and debugging are crucial to detect errors, bottlenecks, and failures. *LLMs can process logs in real time, identify issues, trigger alarms, and suggest corrective actions. In dynamic environments where new TSN flows are added or existing ones removed, the LLM assists in runtime reconfiguration.*

In summary, this paper provides a sequential orchestration workflow in which each block builds on the previous one, while results from the schedulability and simulation stages feed back into earlier decisions for refinement and improvement. This iterative design ensures compliance with QoS requirements while reducing manual effort and increasing scalability.

IV. INITIAL CASE STUDY: PROOF-OF-CONCEPT

To illustrate the potential of LLMs for TSN orchestration, we conducted an initial proof-of-concept study using several state-of-the-art models. In this study, we evaluate the models on orchestration steps (B–F) (refer to Fig. 2). We only cover steps B–F to narrow the focus of the framework, making it easier to provide readers with an intuitive understanding of how current LLMs behave when tasked with TSN-related orchestration. The prompt was designed to cover orchestration steps (B), (C), (D), (E), and (F) in Fig. 1. This section provides an illustrative example that complements the overall vision presented in Fig. 1. We consider four progressively complex test cases: (1) three end stations connected via two switches (refer to Fig. 3a) with three flows, and Erdos Renyi Graph (ERG) [14] topologies (refer to Fig. 3b) with (2) five flows, (3) ten flows, and (4) twenty flows. All experiments were conducted on a Windows laptop equipped with an Intel® Processor Core™ i7-10610U CPU running at 1.80GHz, and 32 GB RAM. Each case study is evaluated using selected state-of-the-art models from Table I. For consistency, all models are prompted with identical input descriptions, and their responses are compared in terms of OMNeT++ configuration correctness, completeness of outputs, and traffic type identification. We verify the syntax correctness and completeness of the OMNeT++ simulation files.

Model	Company	Access Mode	Token Usage	Cost (USD)	OMNeT++ Config Accuracy	Output Completeness	Traffic Type Identification
GPT-5	OpenAI	API	77699	\$ 0.742	Partially Correct	Partially Complete	TT
GPT-4o	OpenAI	API	8419	\$ 0.054	Incorrect	Incomplete	Not Identified
GPT-4o-mini	OpenAI	API	9290	\$ 0.004	Incorrect	Partially Complete	Not Identified
GPT-o3	OpenAI	API	24042	\$ 0.168	Incorrect	Partially Complete	TT, AVB Class A, AVB Class B
GPT-o1-mini	OpenAI	API	18966	\$ 0.069	Incorrect	Partially Complete	Not Identified
Claude 3.7	Anthropic	API	9361	\$ 0.428	Not Attempted	Incomplete	Not Identified
Gemini 2.5 Pro (gemini-2.5-pro)	Google DeepMind	API	25115	\$ 0.211	Partially Correct	Partially Complete	TT
Grok 4	xAI	API	20568	\$ 0.229	Incorrect	Incomplete	TT
Mistral Large	Mistral AI	API	17054	\$ 0.083	Incorrect	Partially Complete	TT, AVB Class A, AVB Class B
LLaMA 3.1-70B	Meta	Self Host (Ollama)	NA	NA	Incorrect	Incomplete	Priorities without Traffic Types

OMNeT++ Config Accuracy: *Partially Correct*: Model generated both .ned and .ini files with syntax/parameter errors. *Incorrect*: Files missing, incomplete, hallucinated, or contained major invalid syntax. *Not Attempted*: Model did not try to generate simulation files.

Output Completeness: *Complete*: Full answer provided with all required fields. *Partially Complete*: Some fields missing or truncated. *Incomplete*: Major parts missing or answer cut off.

Traffic Type Identification: Whether the model explicitly assigned flows to TSN traffic types (TT, AVB Class A, AVB Class B, etc.). *Not Identified* means no traffic types are assigned to the flows. *Priorities without Traffic Types* means the priorities are assigned to the flows but the traffic types are not mentioned.

Table I: Illustrative case study for TSN orchestration, highlighting token usage, cost, and output quality without fine-tuning.

A. Prompt

We present a sample prompt used for the experiments. The files *topology.txt* and *flows.txt* contain the network topology and flow information, respectively, provided in a structured format.

Example Prompt used in the case study

You are a TSN orchestrator. Suggest the TSN architecture, traffic type, and traffic priority for this network.

Topology: *topology.txt* Links: 1 Gbps, 5 μ s propagation delay, cut-through switches.

Flows: *flows.txt*

Constraints:

Ensure deadlines are satisfied. Ensure Quality of Service is met. The topology is given as text. Switch represents the TSN switch. PLC represents the end stations. WIRE represents the connection. Flow information is given as text file.

Output:

Provide the architecture of the TSN network and mention which mechanisms to use. Mention the flow priority. Mention the traffic type of the flow. Perform the scheduling, configuration, and schedulability analysis using mathematical models. Find the worst case delays and jitter. For the generated architecture give the OMNeT++ simulation file for verification. Provide the output in text.

B. Outcomes

Several insights emerge from this experiment and are listed as follows:

- **Latency:** Latency is an important parameter when using LLMs. API-based models such as GPT-5, GPT-4o, Claude 3.7, Gemini 2.5 Pro, Grok 4, and Mistral Large returned results with relatively low latency. In contrast, the LLaMA 3.1-70B model, hosted locally via Ollama, exhibited significantly higher response times (refer to

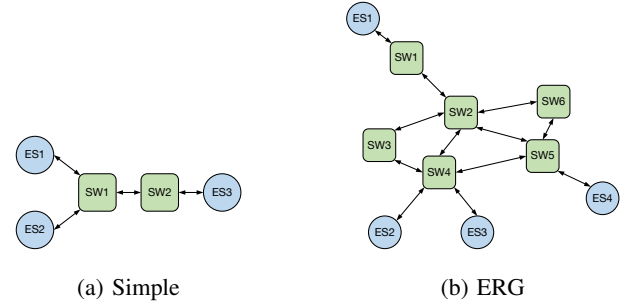


Fig. 3: Test Topologies

Fig. 4). This illustrates the tradeoff between hosting models locally and using commercial APIs.

- **Cost efficiency:** Token usage and costs vary widely. As shown in Table I, GPT-4o-mini offers the lowest cost per run, while GPT-5, followed by Claude 3.7, is substantially more expensive. These numbers reflect raw token costs for input/output and does not include expenses for fine-tuning or specialized hosting.
- **OMNeT++ Configuration Accuracy:** The OMNeT++ configuration files generated by the models should ideally be ready for direct use in the simulator without any manual intervention. However, the off-the-shelf models tested with the given input prompt did not generate complete, simulation-ready configuration files (refer to Table I). Models classified as **Partially Correct** produced the .ned and .ini files, but the syntax was partially correct or several parameters were missing. Models that generated only a few lines of the .ned and .ini files, or hallucinated entirely irrelevant text, were classified as **Incorrect**. Interestingly, Claude 3.7 did not produce

any OMNeT++ files and was therefore classified as **Not Attempted**.

- **Output Completeness:** The quality of generated outputs varied significantly across models (refer to Table I). Output quality was judged based on the ability of the model to produce complete responses for a given prompt. In general, most models produced syntactically coherent configurations that were aligned with the task description. However, certain models, such as Claude 3.7 and LLaMA 3.1-70B, often generated incomplete responses, with outputs terminating abruptly.
- **Traffic Type Identification:** Models such as GPT-5, Gemini 2.5 Pro, and Grok 4 assigned all flows as TT flows. However, models such as GPT-4o, GPT-4o-mini, and Claude 3.7 did not provide any traffic type assignment (refer to Table I). Interestingly, LLaMA 3.1-70B assigned priorities to the flows but did not specify the traffic type. Overall, the lack of explicit traffic type assignments in models limits their usefulness for TSN orchestration.

C. Takeaway

This case study illustrates that current off-the-shelf LLMs are capable of parsing natural-language TSN orchestration tasks, however, the outputs reveal several key limitations, including substantial differences in variability in output quality, incorrect or incomplete OMNeT++ configuration files, and hallucinated traffic type assignments. *Prompt design:* The effectiveness of an LLM in performing a task depends heavily on how the task is described in the prompt [17]. Clear and precise instructions are therefore essential for efficient performance. A critical takeaway is the need for structured prompt templates that effectively guide the LLM in TSN orchestration tasks. Future research should systematically evaluate prompt design and its impact on output accuracy. *TSN benchmark:* Comprehensive evaluation and benchmarking of different LLMs on TSN tasks are necessary. This will clarify which models perform best and are suitable for fine-tuning. It is highly unlikely that all stages of TSN orchestration can be solved by LLMs. Furthermore, some models may excel in specific stages but fail in others. Selecting the best model for each stage therefore requires detailed comparative analysis. Finding the right TSN stage to evaluate is another research direction. *External tools:* The results clearly show that none of the models perform adequately in traffic type and priority assignment. Thus, integrating external tools such as NC and TTA solvers [10] remains essential.

V. LESSONS LEARNED & OPEN RESEARCH DIRECTIONS

A. Benefits of LLM-aided TSN Orchestrator

Configuring TSN networks directly through natural language, without requiring specialized modeling languages such as YANG, RestConf, XML, or JSON, makes LLM-based

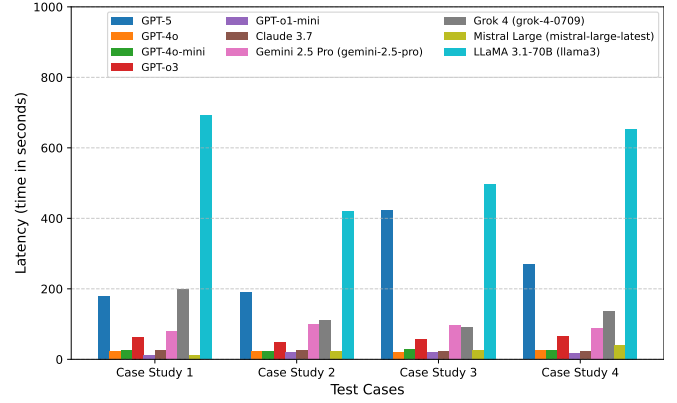


Fig. 4: Latency across different LLM models.

orchestration highly attractive for industry and real-world deployments. The main benefits include:

- **Reduced engineering effort:** LLMs may reduce the time spent on repetitive configuration, translation, and documentation tasks across the TSN workflow, such as GCL configuration.
- **Assistance for complex workflows:** LLMs can help prepare inputs, such as converting the inputs from one format to another format, summarize outputs, and coordinate validation steps for complex TSN configurations. However, the final verification should remain with established state-of-the-art tools and engineering review by domain experts.
- **User-friendly system:** Non-experts can design deterministic TSN networks through natural language interfaces, making the system more accessible.

B. Challenges and Open Issues

Despite these advantages, several challenges remain for realizing an LLM-aided TSN orchestrator:

- **Mathematical module:** TSN WCD analysis relies on mathematical tools such as NC, which LLMs cannot perform reliably. Therefore, integrating dedicated computational modules and optimization solvers, as suggested in [6], is necessary. Integrating LLMs with existing NC tools will provide the necessary mathematical guarantees for WCD analysis.
- **TSN datasets:** The absence of open-source datasets and algorithms limits the ability to fine-tune and verify models for TSN-specific tasks. Having open-source TSN test cases for different applications supports reproducibility.
- **Continuous verification:** Misconfigurations in TSN can cause wasted resources, QoS violations, or safety-critical failures. LLM outputs therefore require continuous verification and validation, with the help of NC solvers, such as the ones provided in [9], [18], and existing TSN configuration tools, ideally with minimal human intervention.

- **Assurance and trust:** Current off-the-shelf models are not yet sufficient to replace the existing mathematical tools in assurance-relevant stages (C-E). For these stages, LLMs can assist in input preparation, output translation, report generation, and assistance of established tools. Future LLM-assisted TSN systems should move from one-shot generation toward hybrid workflows that combine LLMs with solvers, simulators, validators, and, where relevant, model checkers or theorem provers.

C. Integration with External Tools

Hybrid integration with established tools is essential for practical deployment. Since LLMs perform poorly in mathematical reasoning [6], tasks such as GCL generation (D), network configuration (C-D), schedulability analysis (E), WCD calculation (E), and TTA (C) should be handled or verified by external tools. Here we propose using NC tools for (E), such as those proposed in [9], [18], heuristics and mathematical algorithms to generate the GCL for (D), such as [1], [8], [19], metaheuristics and deep reinforcement learning (DRL)-based solutions for TTA, as proposed by [2], [10], and OMNeT++ and NS-3 based simulation frameworks for (F-G), as provided by INET and [11], [12], [14].

VI. CONCLUSION AND FUTURE DIRECTIONS

In this paper, we present a vision for Large Language Model (LLM)-assisted orchestration of Time-Sensitive Networking (TSN). We outline an end-to-end workflow that includes input parsing, traffic type assignment, scheduling and configuration, schedulability analysis, OMNeT++ file generation, simulation-based validation, log analysis, and monitoring. Unlike general network automation, TSN imposes deterministic guarantees and mixed-criticality constraints, making orchestration uniquely challenging. We argue that LLMs can serve as an automation layer when complemented by external tools such as NC engines and advanced scheduling algorithms. We also showcase illustrative case studies that examine the capabilities of existing off-the-shelf models. The results suggest that current off-the-shelf LLMs can already assist with selected TSN tasks, especially parsing structured prompts and generating preliminary configuration artifacts, though with clear tradeoffs in latency, cost, and output quality. However, these outputs are not yet sufficient as standalone assurance evidence. A more promising direction is hybrid orchestration, in which LLMs prepare inputs, coordinate tools, and summarize results, while external configuration, schedulability, simulation, and formal verification tools provide the evidence used for timing and deployment decisions. Looking ahead, we identify four research directions: (i) developing open TSN datasets and standardized benchmarks for checkable TSN tasks, (ii) fine-tuning or adapting LLMs for TSN-specific artifacts and workflows, (iii) building hybrid orchestration systems that integrate LLMs with external configuration, schedulability, simulation,

and formal verification tools, and (iv) defining explicit assurance arguments for how LLM-generated artifacts are validated before deployment.

REFERENCES

- [1] R. Serna Oliver, S. S. Craciunas, and W. Steiner, "IEEE 802.1Qbv Gate Control List Synthesis Using Array Theory Encoding," in *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2018, pp. 13–24.
- [2] V. Gavriluț and P. Pop, "Traffic-Type Assignment for TSN-Based Mixed-Criticality Cyber-Physical Systems," *ACM Trans. Cyber-Phys. Syst.*, vol. 4, no. 2, jan 2020.
- [3] J. Zhang, C. Petrucci, K. Nikolić, and F. Tramèr, "RealMath: A Continuous Benchmark for Evaluating Language Models on Research-Level Mathematics," 2025. [Online]. Available: <https://arxiv.org/abs/2505.12575>
- [4] O. G. Lira, O. M. Caicedo, and N. L. S. da Fonseca, "Large Language Models for Zero Touch Network Configuration Management," *IEEE Communications Magazine*, vol. 63, no. 7, pp. 146–153, 2025.
- [5] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?" *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, Jun. 2024.
- [6] Y. Huang, H. Du, X. Zhang, D. Niyato, J. Kang, Z. Xiong, S. Wang, and T. Huang, "Large Language Models for Networking: Applications, Enabling Techniques, and Challenges," *IEEE Network*, vol. 39, no. 1, pp. 235–242, 2025.
- [7] K. B. Kan, H. Mun, G. Cao, and Y. Lee, "Mobile-LLaMA: Instruction Fine-Tuning Open-Source LLM for Network Analysis in 5G Networks," *IEEE Network*, vol. 38, no. 5, pp. 76–83, 2024.
- [8] R. Debnath, M. Barzegaran, and S. Steinhorst, "Towards an Optimized Multi-Cyclic Queuing and Forwarding in Time Sensitive Networking With Time Injection," *IEEE Internet of Things Journal*, pp. 1–18, 2025.
- [9] L. Zhao, P. Pop, and S. Steinhorst, "Quantitative Performance Comparison of Various Traffic Shapers in Time-Sensitive Networking," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2899–2928, 2022.
- [10] R. Debnath, L. Zhao, and S. Steinhorst, "Learning-Based Traffic Classification for Mixed-Critical Flows in Time-Sensitive Networking," in *ICC 2025 - IEEE International Conference on Communications*, 2025, pp. 5926–5932.
- [11] R. Debnath, P. Hortig, L. Zhao, and S. Steinhorst, "Advanced Modeling and Analysis of Individual and Combined TSN Shapers in OMNeT++," in *2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2023, pp. 176–185.
- [12] —, "Quantifying the Impact of Frame Preemption on Combined TSN Shapers," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, 2024, pp. 1–9.
- [13] P. Graydon and I. Bate, "Safety Assurance Driven Problem Formulation for Mixed-Criticality Scheduling," in *1st International Workshop on Mixed Criticality Systems*, 2013, pp. 19–24.
- [14] R. Debnath, L. Zhao, M. Barzegaran, and S. Steinhorst, "CyclicSim: Comprehensive Evaluation of Cyclic Shapers in Time-Sensitive Networking," in *2025 IEEE 22nd Consumer Communications & Networking Conference (CCNC)*, 2025, pp. 01–09.
- [15] C. Liu, X. Xie, X. Zhang, and Y. Cui, "Large Language Models for Networking: Workflow, Advances and Challenges," *IEEE Network*, 2024.
- [16] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?" in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, ser. HotNets '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 189–195.
- [17] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A Survey of Large Language Models," 2025. [Online]. Available: <https://arxiv.org/abs/2303.18223>
- [18] L. Zhao, P. Pop, and S. S. Craciunas, "Worst-Case Latency Analysis for IEEE 802.1Qbv Time Sensitive Networks Using Network Calculus," *IEEE Access*, vol. 6, pp. 41 803–41 815, 2018.
- [19] N. Reusch, L. Zhao, S. S. Craciunas, and P. Pop, "Window-Based Schedule Synthesis for Industrial IEEE 802.1Qbv TSN Networks," in *2020 16th IEEE International Conference on Factory Communication Systems (WFCS)*, 2020, pp. 1–4.